

```

1  public class ListHw {
2
3      public static void main(String[] args) {
4          int [] arr = {2, 5, 1, 7, -4, 6, -1, 12, 3, 9, 11, 4, 7, -2, 11, -2};
5          Node<Integer> lst = NodeTest.build(arr);
6          NodeTest.show(lst);
7
8          System.out.println(sequenceCount(lst));
9          System.out.println(longestSequence(lst));
10
11         NodeTest.show(getSequenceList(lst));
12         NodeTest.show(getSequenceSumList(lst));
13
14         /*
15         [ 2, 5, 1, 7, -4, 6, -1, 12, 3, 9, 11, 4, 7, -2, 11, -2]
16         4
17         6
18         [ 4, 1, 6, 1]
19         [ 15, 6, 46, 11]
20         */
21
22         int[] arr2 = {2, 5, 7, 8, -3, -1, 0, 2, 4, 6, 5, 5, 6, 7, 12};
23         Node<Integer> lst2 = NodeTest.build(arr2);
24         System.out.println(longestUpSequence(lst2));
25         // 6
26     }
27
28     // question 1 - 1
29     public static int sequenceCount(Node<Integer> lst){
30         int count = 0;
31         boolean sequence = true;
32         Node<Integer> pos = lst;
33         while (pos != null)
34         {
35             if (pos.getValue() < 0 && sequence){
36                 count++;
37                 sequence = false;
38             } else sequence = true;
39             pos = pos.getNext();
40         }
41         return count;
42     }
43
44     // question 1 - 2
45     public static int longestSequence(Node<Integer> lst){
46         int count = 0;
47         int longestSequence = 0;
48         Node<Integer> pos = lst;
49         while (pos != null)
50         {
51             if (pos.getValue() > 0){
52                 count++;
53             } else {

```

```

54         if (count > longestSequence)
55             longestSequence = count;
56         count = 0;
57     }
58     pos = pos.getNext();
59 }
60 return longestSequence;
61 }
62
63 // question 1 - 3
64 public static Node<Integer> getSequenceList(Node<Integer> lst){
65     int count = 0;
66     Node<Integer> temp = new Node<>(-1);
67     Node<Integer> pos2 = temp;
68     Node<Integer> pos = lst;
69     while (pos != null)
70     {
71         if (pos.getValue() > 0){
72             count++;
73         } else {
74             pos2.setNext(new Node<>(count));
75             pos2 = pos2.getNext();
76             count = 0;
77         }
78         pos = pos.getNext();
79     }
80     return temp.getNext();
81 }
82
83 // question 1 - 4
84 public static Node<Integer> getSequenceSumList(Node<Integer> lst){
85     int count = 0;
86     Node<Integer> temp = new Node<>(-1);
87     Node<Integer> pos2 = temp;
88     Node<Integer> pos = lst;
89     while (pos != null)
90     {
91         if (pos.getValue() > 0){
92             count += pos.getValue();
93         } else {
94             pos2.setNext(new Node<>(count));
95             pos2 = pos2.getNext();
96             count = 0;
97         }
98         pos = pos.getNext();
99     }
100    return temp.getNext();
101 }
102
103
104 public static int longestUpSequence(Node<Integer> lst) {
105     int maxCount = 0;
106     int currentCount = 0;

```

```
107
108     if (lst.getValue() > 0) {
109         currentCount = 1;
110         maxCount = 1;
111     }
112
113     Node<Integer> prev = lst;
114     Node<Integer> pos = lst.getNext();
115     while (pos != null) {
116         if (pos.getValue() > 0) {
117             if (pos.getValue() > prev.getValue())
118                 currentCount++;
119         } else
120             currentCount = 0;
121
122         if (currentCount > maxCount) {
123             maxCount = currentCount;
124         }
125
126         prev = pos;
127         pos = pos.getNext();
128     }
129
130     return maxCount;
131 }
132
133 }
134
```